

Qwen3-Coder-Next Tier 3

Implementation Master Roadmap

From Local Coding Model to Near-Codex Autonomous Software Engineering System

Implementation blueprint | Architecture already finalized | No code, no low-level specs

What this document is

A master implementation roadmap that bridges the finalized architecture and the later deep-dive engineering PDFs. It explains the order of work, the purpose of each stage, the conceptual workflow, the unlocks, and the dependency chain.

Scope

Implementation progression, milestone sequencing, conceptual workflows, capability unlocks, and the logic connecting each stage to the next.

Non-scope

Full code, exhaustive technical specs, and low-level implementation detail. Those belong in separate PDFs later.

Reference target: a fully local, self-hosted, open-source autonomous software engineering system built around Qwen3-Coder-Next.

Roadmap at a Glance

Part	Milestone	Why it matters
1	Foundation	Creates the local runtime, orchestration shell, and clean execution baseline.
2	Filesystem + Local Tooling	Lets the system read, write, and execute against a workspace.
3	Planning Layer	Turns raw requests into ordered work.
4	Research Layer	Supplies supporting context before code changes begin.
5	Memory System	Preserves decisions and project history across sessions.
6	Testing & Review	Adds quality gates so code is not merely plausible.
7	Evaluation Layer	Checks whether the solution answers the actual request.
8	Failure Recovery	Keeps the system stable when tools or assumptions fail.
9	Repository Intelligence	Builds a usable map of the codebase.
10	Code Knowledge Graph	Adds relationship-aware navigation and impact analysis.
11	Context Compression	Keeps the model inside its usable context budget.
12	Multi-Agent Coordination	Makes the specialized agents behave like one system.
13	Benchmark Harness	Turns progress into measured improvement.
14	Deployment & Human Approval	Creates a safe path from code to release.
15	Near-Codex Integrated System	Unifies the stack into an autonomous workflow.

Key idea

Each stage should produce a working system. The roadmap is intentionally incremental so the project compounds instead of becoming a glamorous pile of partially wired ideas.

PART 1 - Foundation

Goal

Establish the local execution baseline so the system behaves like software, not a prompt with ambition.

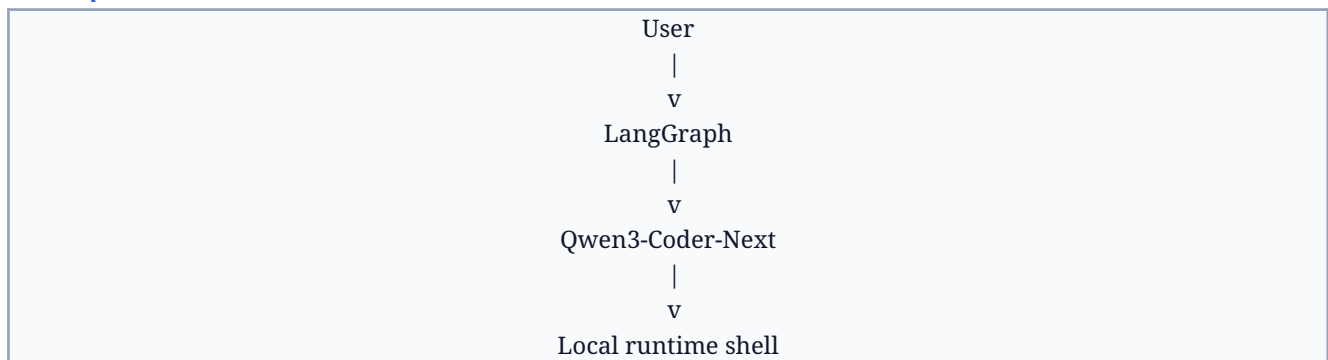
Components Built

- Qwen3-Coder-Next runtime hookup
- LangGraph orchestration shell
- Configuration, logging, and run-state conventions
- Artifact and trace storage
- Minimal response contract for tasks

Why This Stage Exists

Before any agent logic is useful, the project needs a stable place to run, a predictable task interface, and a way to observe what happened.

Conceptual Workflow



Internal Interactions

The orchestrator owns the task state, the model produces decisions or draft outputs, and the runtime records each step so later stages can reuse the same execution contract.

Capabilities Unlocked

- Accepts a task and returns a structured result.
- Provides a clean base for later agents and tools.
- Creates repeatable execution traces.

Exit Criteria

- System boots locally and completes a full task loop without manual repair.
- Task state, logs, and artifacts persist across runs.
- The response contract stays stable enough for later agents to consume.
- Later stages can attach without redesigning the runtime shell.

Limitations Remaining

- No tool use beyond the core runtime.
- No persistent memory.
- No real planning, testing, or repo awareness.

Why The Next Part Is Needed

The next stage is needed because a model without a workspace is still just a very talkative paperweight.

PART 2 - Filesystem + Local Tooling

Goal

Give the system safe access to the local repository, shell, and workspace so it can inspect and modify real code.

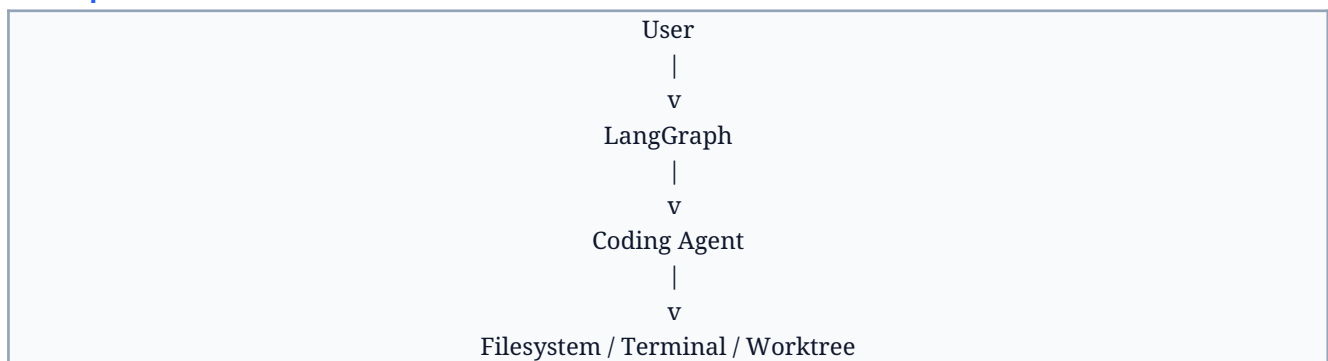
Components Built

- Filesystem read/write layer
- Terminal or shell tool access
- Workspace conventions
- Initial worktree isolation pattern
- Basic repository navigation utilities

Why This Stage Exists

Coding begins only when the system can touch files, inspect project structure, and run local commands in a controlled space.

Conceptual Workflow



Internal Interactions

The coding path becomes grounded in the workspace, while the isolation layer ensures risky edits stay away from the main branch until proven safe.

Capabilities Unlocked

- Reads and edits local files.
- Runs commands against a sandboxed workspace.
- Prepares the ground for isolated experimentation.

Exit Criteria

- The system can inspect, read, and edit files inside a controlled workspace.
- Local commands and tests run inside the sandboxed environment.
- Worktree isolation protects the main branch from failed experiments.
- Repository navigation is reliable enough for planning and research handoff.

Limitations Remaining

- No structured planning yet.
- No research loop.
- No memory, review, or evaluation gates.

Why The Next Part Is Needed

Planning must come next, because letting an agent freestyle through a codebase is how you get entropy with a commit hash.

PART 3 - Planning Layer

Goal

Convert a user request into an ordered execution path instead of throwing the full problem at the coding model at once.

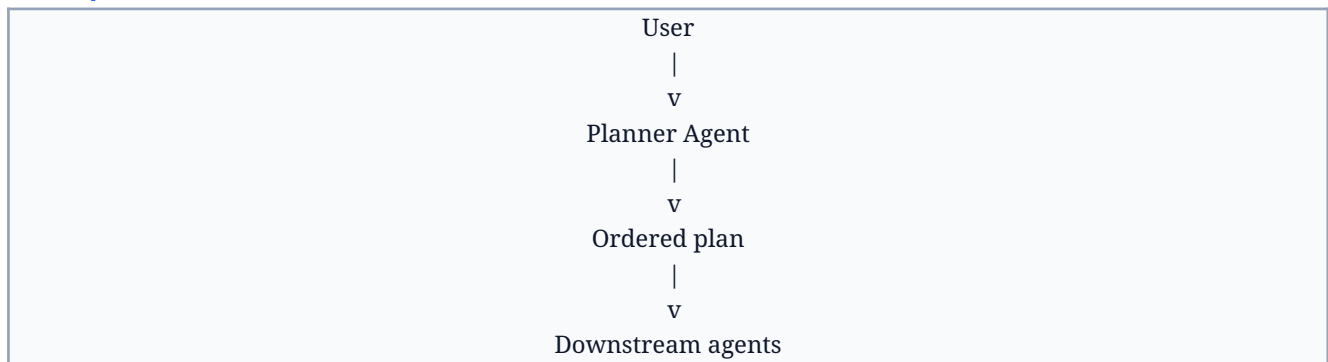
Components Built

- Planner Agent
- Task decomposition logic
- Step ordering and dependency tracking
- Plan state shared with later agents

Why This Stage Exists

Large tasks fail when the system jumps straight into code generation. Planning reduces chaos and keeps the workflow intelligible.

Conceptual Workflow



Internal Interactions

The planner writes the sequence of work, marks dependencies, and hands a constrained objective to coding and research so they stay aligned.

Capabilities Unlocked

- Breaks goals into manageable steps.
- Prevents random tool calls.
- Creates a stable handoff to research and coding.

Exit Criteria

- A feature request can be decomposed into ordered, dependency-aware steps.
- The planner can distinguish prerequisites from downstream work.
- The plan can be handed to coding and research without extra human reshaping.
- Minor context changes do not collapse the plan into noise.

Limitations Remaining

- Still lacks authoritative external context.
- Still cannot remember past decisions reliably.
- Still does not know whether the plan is actually correct.

Why The Next Part Is Needed

Research is required so the planner and coder stop guessing when the task depends on docs, APIs, or repository facts.

PART 4 - Research Layer

Goal

Collect the information needed to execute accurately before code is written.

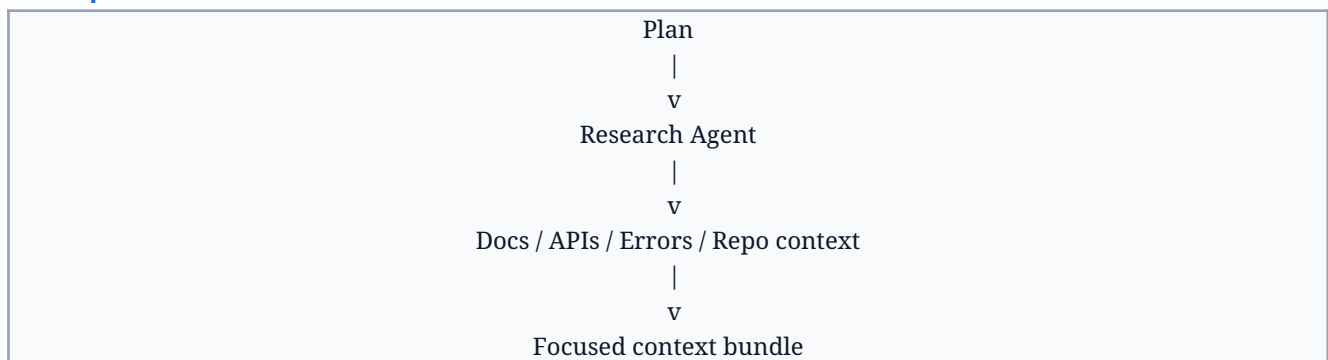
Components Built

- Research Agent
- Documentation and API reading flow
- Error and issue investigation flow
- Context packaging for downstream agents

Why This Stage Exists

Many failures come from insufficient context, not bad code generation. Research gives the system a way to resolve ambiguity early.

Conceptual Workflow



Internal Interactions

The research agent narrows the search space, extracts relevant facts, and returns only the useful context so coding does not inherit the full mess.

Capabilities Unlocked

- Reads supporting material.
- Investigates unclear requirements or failures.
- Supplies clean context to the coder.

Exit Criteria

- Relevant docs, APIs, and issue trails can be retrieved for a task.
- The research output is concise enough for downstream agents to use directly.
- Noise can be filtered out in favor of task-relevant facts.
- Coding can proceed with reduced uncertainty instead of guesswork.

Limitations Remaining

- No durable history of what was learned.
- No automated quality judgment.
- No recovery strategy if research or coding fails.

Why The Next Part Is Needed

A memory system is needed so the agent stops relearning the same lessons like an expensive goldfish.

PART 5 - Memory System

Goal

Preserve useful information across turns, sessions, and projects so the system behaves like a continuing engineering process.

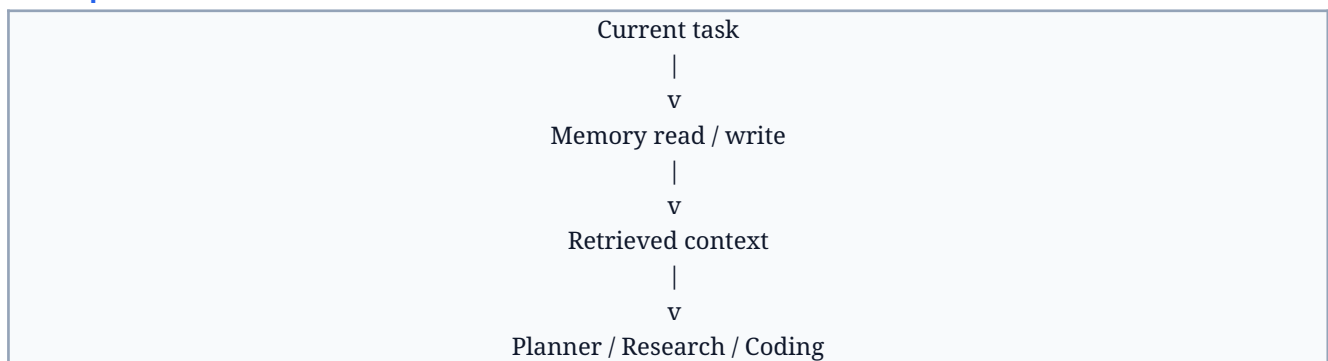
Components Built

- Working memory
- Session memory
- Project memory
- Global memory
- Qdrant-backed retrieval and summarization policy

Why This Stage Exists

Without memory, every run starts from zero and the project becomes a loop of repeated explanations, repeated mistakes, and repeated regret.

Conceptual Workflow



Internal Interactions

Short-lived state stays close to the orchestrator, while persistent layers store project decisions, preferences, and durable lessons for future reuse.

Capabilities Unlocked

- Remembers project conventions.
- Retains prior decisions and fixes.
- Keeps repeated tasks faster and more consistent.

Exit Criteria

- Architecture decisions persist across sessions and restarts.
- Previous bug fixes and known patterns can be retrieved when needed.
- Session memory survives interruption without losing current work context.
- Retrieval quality is good enough to support repeated project work.

Limitations Remaining

- Memory is not yet protected by quality gates.
- The system can still produce incorrect code with confidence, which is a disturbingly common talent.

Why The Next Part Is Needed

Testing and review must follow, because memory without validation just preserves mistakes with better branding.

PART 6 - Testing & Review

Goal

Add correctness and quality gates so generated changes are checked before anyone pretends they are done.

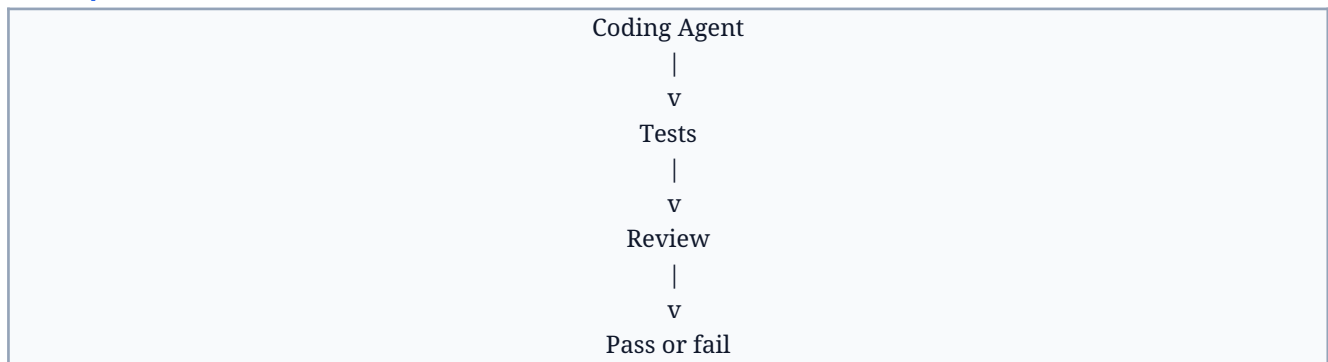
Components Built

- Testing Agent
- Review Agent
- Automated test execution flow
- Code quality and security review pass

Why This Stage Exists

A coding agent needs friction. Otherwise it happily emits plausible nonsense and leaves humans to discover the mess later.

Conceptual Workflow



Internal Interactions

The testing agent proves the code runs, and the review agent checks whether it is readable, maintainable, and safe enough to keep moving.

Capabilities Unlocked

- Runs test suites.
- Catches obvious regressions.
- Reviews design and implementation quality.

Exit Criteria

- Generated code can be executed against automated tests and reviewed for quality.
- Test failures produce actionable root-cause signals instead of raw noise.
- Review gates catch obvious security, correctness, and maintainability issues.
- Only code that clears both test and review gates advances forward.

Limitations Remaining

- Passing tests still does not guarantee the right solution.
- Failures still need a structured interpretation layer.

Why The Next Part Is Needed

Evaluation is needed because a system can pass tests and still solve the wrong problem, which is a very human style of success.

PART 7 - Evaluation Layer

Goal

Compare the implementation against the original request and reject solutions that are technically valid but semantically off-target.

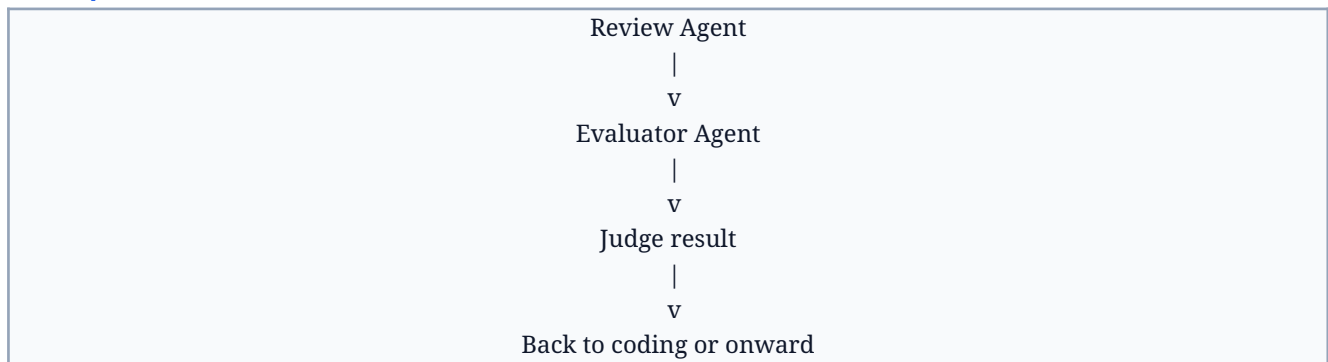
Components Built

- Evaluator / Judge Agent
- Requirement alignment checks
- Quality scoring
- Structured rejection feedback

Why This Stage Exists

Tests verify behavior, not intent. The evaluator closes the gap between 'works' and 'works for the right reason'.

Conceptual Workflow



Internal Interactions

The evaluator compares output against the plan and task intent, then either approves progression or sends actionable feedback back into the loop.

Capabilities Unlocked

- Detects wrong-problem solutions.
- Scores completeness and alignment.
- Improves final quality before deployment.

Exit Criteria

- The evaluator can tell the difference between correct code and the wrong solution.
- Implementation is compared against the original task spec and acceptance criteria.
- Rejected outputs produce structured feedback that the coder can actually use.
- Semantic alignment is verified before anything reaches deployment.

Limitations Remaining

- Still depends on stable failure handling.
- Still needs recovery when the evaluator rejects the work.

Why The Next Part Is Needed

A recovery layer must follow so rejection does not turn into a loop of panic, retries, and existential collapse.

PART 8 - Failure Recovery

Goal

Replace blind retry behavior with diagnosis, strategy selection, and safe continuation.

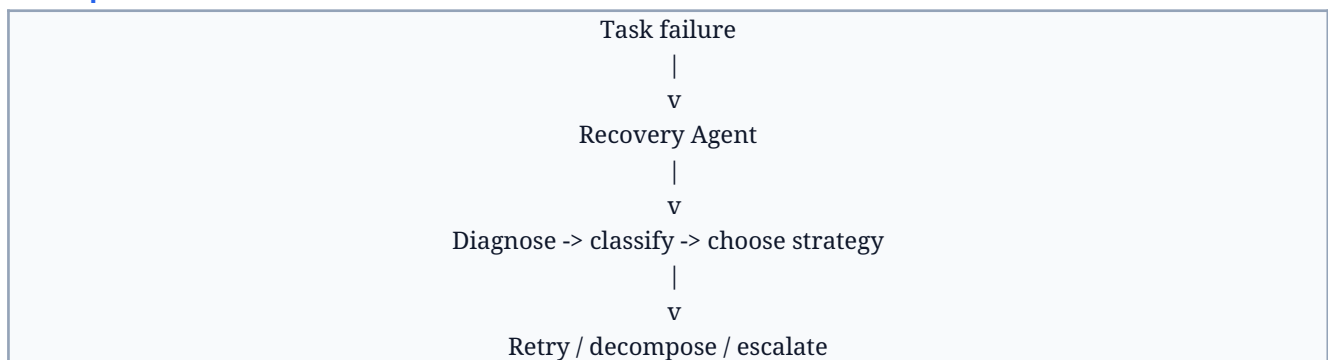
Components Built

- Failure Recovery Agent
- Failure taxonomy
- Retry strategy library
- Escalation and safe-abort logic

Why This Stage Exists

At scale, failures are normal. What matters is whether the system can interpret them instead of thrashing helplessly.

Conceptual Workflow



Internal Interactions

The recovery agent classifies the failure, chooses a response, and routes the system back to progress without corrupting state.

Capabilities Unlocked

- Handles transient tool failures.
- Retries with more context when needed.
- Escalates ambiguous cases safely.

Exit Criteria

- Common transient failures are classified instead of blindly retried.
- The recovery path can choose retry, decompose, alternate, escalate, or abort safely.
- Failed runs preserve state and do not corrupt the working branch.
- Recovery measurably improves completion rate versus naive retry loops.

Limitations Remaining

- Still lacks deep codebase understanding.
- Still needs better navigation of large repositories.

Why The Next Part Is Needed

Repository intelligence comes next so the system can reason about where code lives and how the project is shaped.

PART 9 - Repository Intelligence

Goal

Create a practical understanding of the repository so the agent can move beyond raw file access.

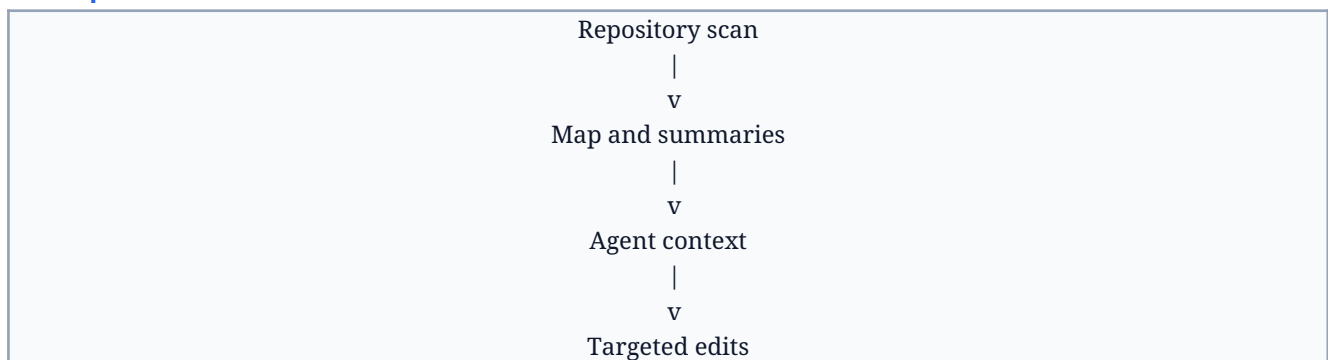
Components Built

- Repository map
- Folder and module summaries
- Dependency overviews
- File purpose summaries

Why This Stage Exists

Large projects are not navigable by file dumping alone. The agent needs a compressed map of the repository's structure and purpose.

Conceptual Workflow



Internal Interactions

The repository intelligence layer provides the agent with a coarse but useful model of the codebase before it opens specific files.

Capabilities Unlocked

- Finds relevant modules quickly.
- Explains what files and folders are for.
- Reduces wasted search and context cost.

Exit Criteria

- The system can build and query a repository map and file summaries.
- It can identify dependencies, module boundaries, and likely relevant files.
- Repo navigation no longer requires loading the entire codebase into context.
- Context selection becomes targeted enough for larger projects.

Limitations Remaining

- It is still largely file-centric.
- It does not yet understand rich relationships such as call paths or blast radius.

Why The Next Part Is Needed

The knowledge graph is needed because structure alone is not enough; relationships are what make large codebases navigable.

PART 10 - Code Knowledge Graph

Goal

Add relationship-aware understanding of the codebase so the system can reason about dependencies instead of just filenames.

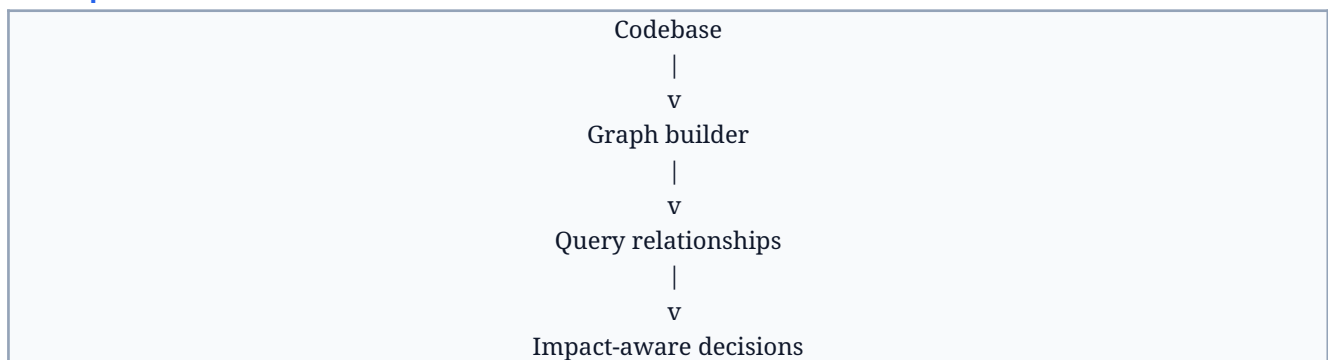
Components Built

- Code Knowledge Graph
- Relationship extraction
- Impact analysis queries
- Call path and dependency views

Why This Stage Exists

As the repository grows, knowing what calls what matters as much as knowing where files are stored.

Conceptual Workflow



Internal Interactions

The graph layer turns code into a navigable web of functions, modules, imports, and dependencies that later agents can query before editing.

Capabilities Unlocked

- Finds call chains.
- Shows what breaks when a function changes.
- Supports dead-code and coupling analysis.

Exit Criteria

- Function and module relationships can answer impact questions before edits happen.
- The graph can trace call paths and dependency chains across the repository.
- Edit planning can use the graph to predict likely breakage points.
- Graph coverage is sufficient for the core areas the agent touches most.

Limitations Remaining

- Graph data can overwhelm the model without compression.
- Relationship awareness still needs to be packaged into usable context.

Why The Next Part Is Needed

Context compression is needed so the graph helps the model instead of drowning it in its own brilliance.

PART 11 - Context Compression

Goal

Keep the system operating inside the model's usable context budget while still preserving the information that matters.

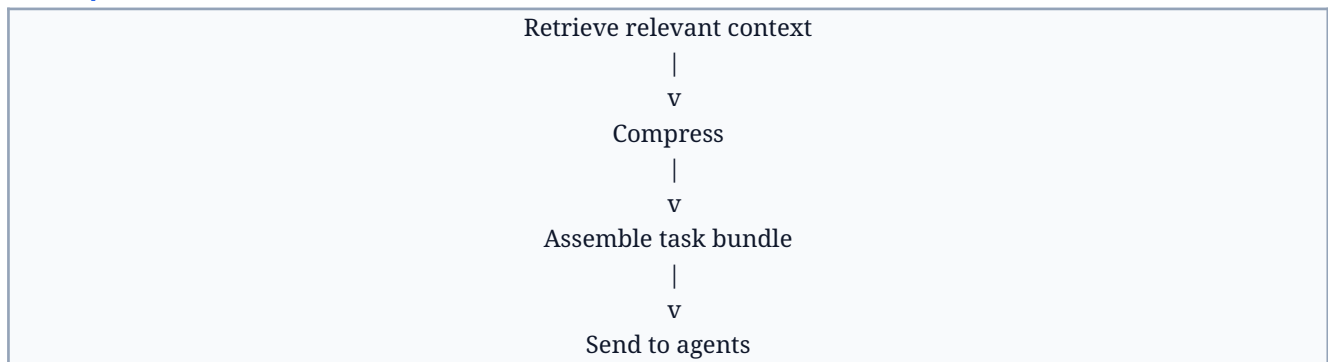
Components Built

- Summarization pipeline
- RAG retrieval policy
- Context packaging and pruning
- Compressed repo and task bundles

Why This Stage Exists

Without compression, large repositories and long task histories quickly exceed what the model can hold in one run.

Conceptual Workflow



Internal Interactions

The compression layer decides what stays explicit, what gets summarized, and what can be fetched again on demand.

Capabilities Unlocked

- Handles larger repositories.
- Keeps task context focused.
- Improves stability across long workflows.

Exit Criteria

- Long repositories can be reduced to task-relevant summaries and slices.
- Retrieved context fits the model window without losing critical dependencies.
- Compression preserves decisions, constraints, and active task state.
- Downstream agents receive concise task packets instead of bloated files.

Limitations Remaining

- Compression alone does not coordinate multiple specialized agents.
- It needs orchestration discipline to stay useful.

Why The Next Part Is Needed

Multi-agent coordination must follow so all the specialized parts act like one system instead of a committee with git access.

PART 12 - Multi-Agent Coordination

Goal

Make planning, research, coding, testing, review, evaluation, and recovery operate as a coherent pipeline.

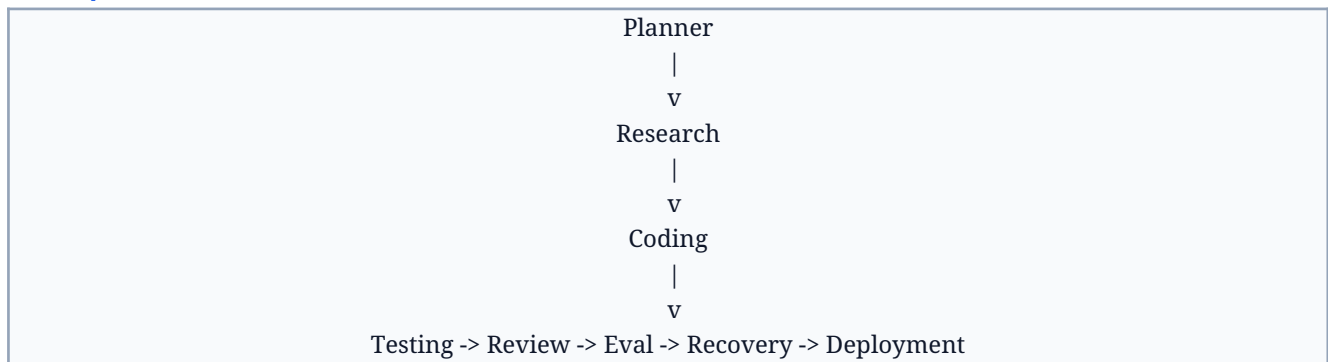
Components Built

- LangGraph routing policies
- State handoff contracts
- Agent specialization boundaries
- Inter-agent progress tracking

Why This Stage Exists

Specialized agents are only valuable when they hand work off cleanly and do not trample each other's context.

Conceptual Workflow



Internal Interactions

The orchestrator becomes the traffic controller, pushing the right state to the right agent at the right time while preserving continuity.

Capabilities Unlocked

- Runs complex workflows end to end.
- Supports parallel or staged execution.
- Reduces context contamination across roles.

Exit Criteria

- Planner, research, coding, testing, review, evaluation, and recovery hand off cleanly.
- Agents can work sequentially or in parallel without overwriting each other.
- The orchestrator can move a task through the full chain without manual glue.
- Multi-step workflows behave like one system instead of a pile of disconnected bots.

Limitations Remaining

- Still needs measurement discipline.
- Still needs a reliable way to tell whether the system is genuinely improving.

Why The Next Part Is Needed

Benchmarking is required so the project stops arguing with itself and starts producing evidence.

PART 13 - Benchmark Harness

Goal

Measure system behavior consistently so every major change can be compared against a baseline.

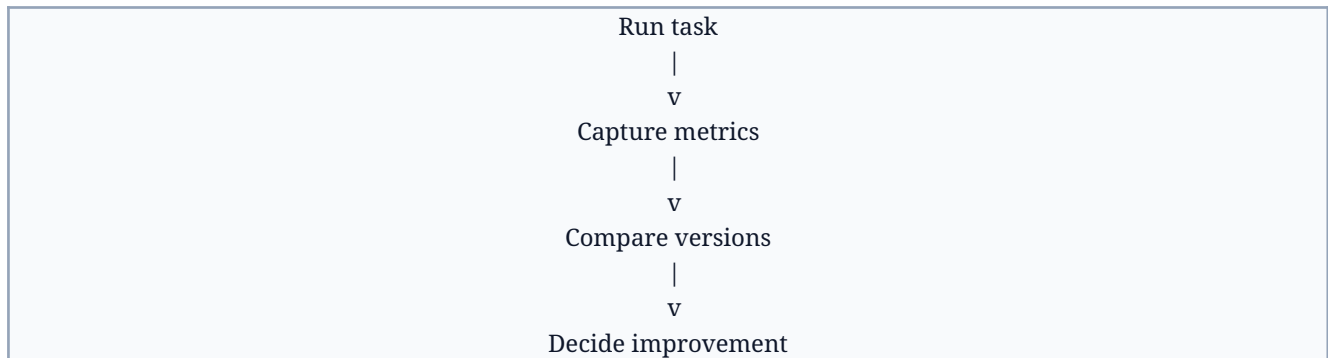
Components Built

- Benchmark Harness
- Run metrics
- Task success tracking
- Regression and quality reporting

Why This Stage Exists

Without metrics, every improvement claim is just a polite hallucination wearing a lab coat.

Conceptual Workflow



Internal Interactions

The harness records performance, quality, and recovery behavior so the team can evaluate the system objectively across releases.

Capabilities Unlocked

- Measures success rate and reliability.
- Tracks cost and latency.
- Exposes regressions early.

Exit Criteria

- Core metrics are captured for success rate, test pass rate, retries, latency, and escalations.
- The system can compare performance across model, prompt, or workflow changes.
- Regression detection is strong enough to block clearly worse builds.
- Improvements are measured rather than guessed.

Limitations Remaining

- Measurement alone does not make the system safe to release.
- Deployment controls still need to be wired in.

Why The Next Part Is Needed

Deployment and human approval are needed so the system can produce artifacts without pretending it should be trusted blindly.

PART 14 - Deployment & Human Approval

Goal

Create a safe release path from validated changes to deployed artifacts with a human gate for high-risk actions.

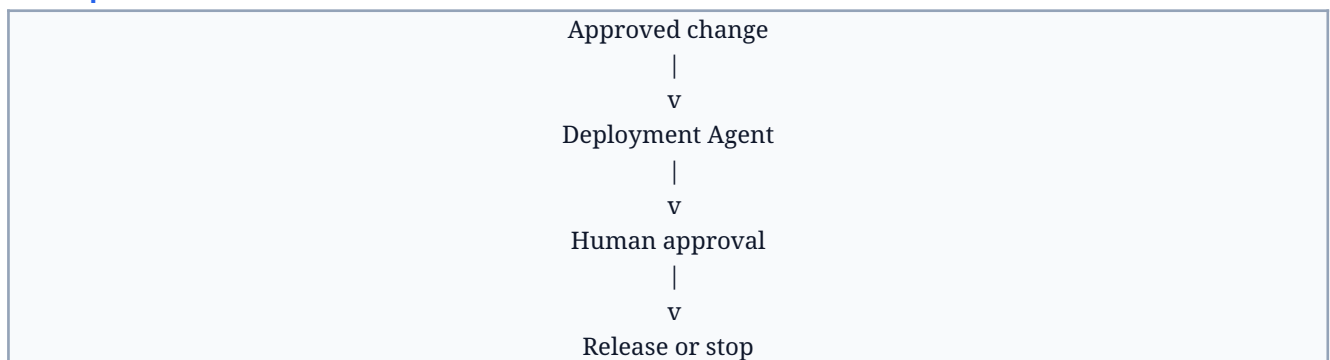
Components Built

- Deployment Agent
- Release packaging flow
- Human approval layer
- Rollback-aware release posture

Why This Stage Exists

Autonomy must stop at the edge of production. Deployment is where enthusiasm meets consequences.

Conceptual Workflow



Internal Interactions

The deployment path handles the mechanics, but the human gate owns the final go or no-go for risky operations.

Capabilities Unlocked

- Packages and publishes artifacts.
- Protects production from impulsive releases.
- Closes the loop from code to deployment.

Exit Criteria

- Build artifacts can be prepared reproducibly for release.
- Deployment actions remain gated behind explicit human approval.
- Safe rollback or no-op behavior exists for uncertain releases.
- The system can hand off a validated release candidate for sign-off.

Limitations Remaining

- Still not the final autonomous system.
- The layers remain partially separate and must be unified.

Why The Next Part Is Needed

The last stage is needed to fuse the whole stack into one operating system for software engineering instead of a pile of decent parts.

PART 15 - Near-Codex Integrated System

Goal

Unify all prior stages into a single autonomous engineering workflow that can plan, research, code, test, review, evaluate, recover, and deploy with human oversight.

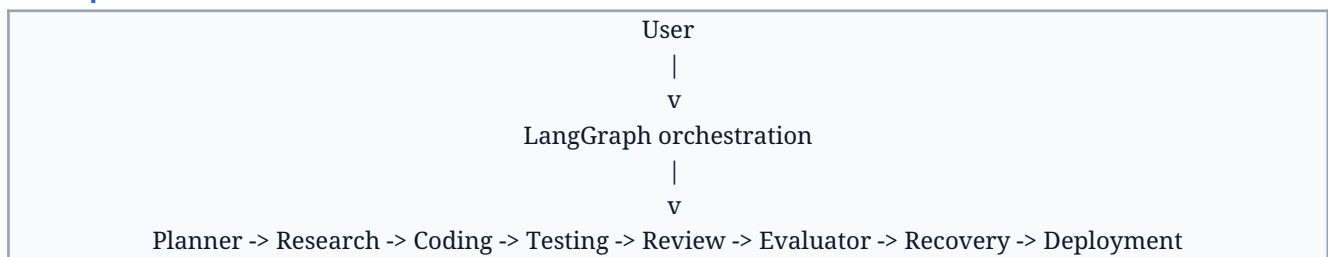
Components Built

- All prior agents and layers
- Orchestration policies
- Repository intelligence and compression
- Benchmark-driven iteration
- Approval-gated deployment

Why This Stage Exists

The project only reaches its real destination when the pieces become one system with stable memory, navigation, quality control, and recovery behavior.

Conceptual Workflow



Internal Interactions

The integrated system uses the same state model, memory policy, and validation loops across all work so each part reinforces the others instead of living as a disconnected feature.

Capabilities Unlocked

- Handles software tasks end to end.
- Maintains continuity across long projects.
- Produces safer and more autonomous outcomes.

Exit Criteria

- The full chain can plan, research, code, test, review, evaluate, recover, and deploy in one flow.
- Memory, repo intelligence, and compression work together across tasks.
- Benchmark data shows stable behavior on representative workloads.
- Human approval remains the final safety boundary before release.

Limitations Remaining

- Still bounded by model quality and environment constraints.
- Still requires human judgment for the riskiest operations.

Why The Next Part Is Needed

There is no next stage in the roadmap. There is only continued refinement, better benchmarks, and fewer surprises.

Final Section

Current Capability vs Final Capability

Capability	Current Baseline	Near-Codex Final State
Task handling	Single request or small coding help.	Multi-stage autonomous execution with planning, research, coding, validation, and deployment.
Memory	Mostly local session state.	Working, session, project, and global memory with persistent project learning.
Codebase understanding	File-level awareness.	Repository intelligence plus relationship-aware graph navigation.
Quality control	Model self-checks and ad hoc review.	Testing, review, evaluator, and benchmark-driven quality gates.
Failure handling	Manual retry or simple error response.	Diagnose, classify, retry, decompose, or escalate safely.
Context management	Prompt stuffing and short summaries.	RAG plus compression that keeps the model focused on relevant context.
Deployment safety	User-managed release decisions.	Approval-gated deployment with controlled release posture.
System maturity	Coding assistant.	Near-Codex autonomous software engineering system.

Build Order Summary

Part 1 -> Foundation
Part 2 -> Filesystem + Local Tooling
Part 3 -> Planning Layer
Part 4 -> Research Layer
Part 5 -> Memory System
Part 6 -> Testing & Review
Part 7 -> Evaluation Layer
Part 8 -> Failure Recovery
Part 9 -> Repository Intelligence
Part 10 -> Code Knowledge Graph
Part 11 -> Context Compression
Part 12 -> Multi-Agent Coordination
Part 13 -> Benchmark Harness
Part 14 -> Deployment & Human Approval
Part 15 -> Near-Codex Integrated System

Implementation principle

Build in layers: first a trustworthy runtime, then local control, then intelligence, then quality gates, then recovery, then large-repo awareness, and finally full orchestration. That sequence keeps the system useful at every milestone instead of turning it into a slow-motion demo.

End of roadmap